ONEHEALTH

Full-Stack Architecture & Repository Reference

Backend (Java/Spring Boot) · Web · Mobile · Admin Portal · Supabase

Developer Handbook — v1.0

August 2026

Table of Contents

1. Introduction & Scope

This booklet is the developer-facing reference for building OneHealth end to end: the Java backend, the React web application, the React Native mobile app, and the React-based admin portal, all backed by Supabase. It complements the SRS by translating the requirements and diagrams already agreed into a concrete repository layout, technology stack, and set of conventions two developers can execute against without re-litigating structural decisions mid-sprint.

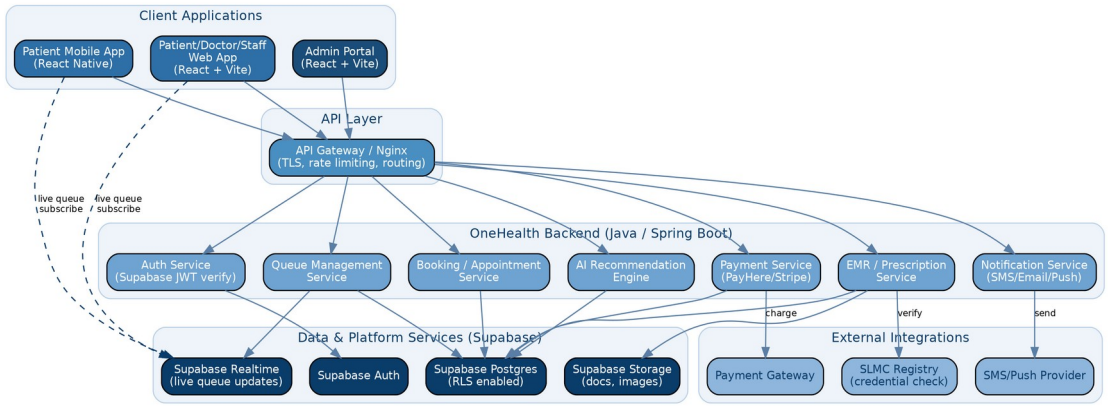
Scope covered here:

- Recommended technology stack per application, with the reasoning for each choice
- System architecture, including how Supabase fits alongside the Java backend
- Full monorepo file/folder structure for backend, web, mobile, admin, and shared packages
- API and real-time communication conventions
- Environments, CI/CD, and a developer onboarding checklist

> **Note:** *This is a living reference. Treat folder names and module boundaries as the default; adjust only with a short note in the repo's ARCHITECTURE.md so both developers stay in sync.*

2. System Architecture Overview

OneHealth uses a client-server architecture with three client applications (Web, Mobile, Admin) talking to a single Java/Spring Boot backend over a REST API, with Supabase providing managed Postgres, authentication, file storage, and real-time pub/sub for the live queue feature. The backend remains the source of business logic (queue rules, credential verification, payments, EMR access control); Supabase is used as managed infrastructure rather than a second application backend.



2.1 Why keep a Java backend if Supabase is the database?

Supabase alone (Postgres + Auto-generated REST/GraphQL + Auth + Realtime) is enough for simple CRUD apps, but OneHealth has domain logic that shouldn't live in the client or in database triggers alone: queue position calculation and re-sequencing, SLMC credential verification workflows, payment reconciliation, prescription/EMR access rules, and the AI recommendation engine. Keeping this in a Spring Boot service layer gives:

- A single, testable place for business rules instead of scattering logic across client apps
- Clean integration points for external systems (payment gateways, SMS, SLMC registry)
- Room to add the AI recommendation engine as a proper service without contorting Postgres functions
- A migration path if you ever need to move off Supabase without rewriting business logic

2.2 What Supabase is responsible for

- Postgres database with Row-Level Security (RLS) as a second line of defence behind the API
- Authentication (email/OTP/social login), issuing JWTs the Spring Boot backend verifies
- Storage buckets for prescription scans, lab reports, doctor credential documents, profile photos
- Realtime channel for live queue position updates pushed straight to Web and Mobile clients

Clients subscribe to Supabase Realtime directly for queue updates (low latency, no backend fan-out needed), while all writes to queue state go through the backend so business rules are enforced consistently.

3. Technology Stack & Justification

Layer	Recommendation	Why it fits a 2-developer team
Backend	Java 21 + Spring Boot 3 (Maven multi-module)	Matches your agreed Java backend; Spring's ecosystem (Security, Data JPA, Validation) reduces boilerplate for auth, queue, payments
Web app	React 18 + Vite + TypeScript	Fast dev server, small config surface, shares component/type conventions with Admin
Mobile app	React Native (Expo) + TypeScript	Reuses React knowledge and a large slice of business logic/hooks with Web; Expo avoids native build overhead for 2 devs
Admin portal	React + Vite + TypeScript (same toolchain as Web)	One toolchain to maintain instead of three; only routes/permissions differ
Styling/UI	Tailwind CSS + a shared component kit	Consistent design system reused across Web/Admin; RN uses NativeWind for the same utility classes
Database	Supabase (managed Postgres)	Removes DB ops burden; built-in Auth/Storage/Realtime cut weeks of boilerplate for a small team
State/data-fetching	TanStack Query + Zustand	Query caching for REST calls; lightweight local state without Redux ceremony
API contract	OpenAPI 3 generated from Spring annotations	Generates typed clients for Web/Mobile/Admin automatically — one contract, three consumers
i18n	i18next (Sinhala, Tamil, English)	Mature, works identically across React and React Native
Realtime	Supabase Realtime (Postgres changes + broadcast)	No separate WebSocket server to build/operate for the live queue
Payments	PayHere (LK-focused) with Stripe as a fallback	PayHere has native LKR/local card support; abstract behind a PaymentProvider interface either way
Notifications	Firebase Cloud Messaging (push) + a local SMS gateway	FCM covers Web+Mobile push in one integration; SMS needed for patients without smartphones
CI/CD	GitHub Actions	Already assumed by your branching/CI annex; free tier is enough at this scale
Hosting	Backend: Fly.io/Render or a small VPS · Web/Admin: Vercel/Netlify · Mobile: EAS Build	Minimal ops overhead; all have generous free/low tiers for an MVP

3.1 Alternatives considered

- Next.js instead of Vite for Web — gives SSR/SEO, but the patient/doctor app is mostly behind auth, so the extra complexity isn't earning its keep yet; keep it as a later option if a public marketing/SEO surface is needed.
- Flutter instead of React Native — strong choice too, but React Native lets both developers reuse React skills and even some hooks/types with the web app, which matters more for a 2-person team.
- Firebase instead of Supabase — Supabase's Postgres + SQL + RLS fits a relational domain like queues/appointments/EMR far better than Firestore's document model, and was already your stated preference.

4. Supabase Integration Strategy

4.1 Auth flow

- Clients authenticate directly against Supabase Auth (email/password or OTP) and receive a Supabase JWT
- Every request to the Spring Boot API carries that JWT; a Spring Security filter verifies it against Supabase's JWKS endpoint
- The backend maps the verified Supabase user id to the application's Patient/Doctor/Staff/Admin role stored in Postgres

4.2 Data access pattern

- Spring Boot uses a service-role Postgres connection (via Spring Data JPA) for all business writes — this is the authoritative write path
- RLS policies are still enabled on every table as defence-in-depth, scoped so anon/authenticated keys used by clients (for realtime subscriptions and storage) can only read what they're entitled to
- Clients never write queue/booking/EMR rows directly through the Supabase client SDK — only read/subscribe

4.3 Storage

- Buckets: doctor-credentials (private), prescriptions (private), lab-reports (private), avatars (public, resized)
- Backend issues short-lived signed URLs for private buckets rather than exposing bucket keys to clients

4.4 Realtime

- A queue_status table (or Postgres NOTIFY-backed channel) is updated by the backend whenever a queue advances
- Web and Mobile subscribe to that table's changes for the dispensary/doctor session the patient is queued at, giving live position updates with no polling

5. Monorepo Structure — Overview

A single monorepo keeps the shared API contract, design tokens, and TypeScript types in one place, which matters a lot for a 2-developer team avoiding version drift across four consumers of the same backend. Turborepo (or plain npm workspaces if you want zero extra tooling) orchestrates the JS/TS apps; the Java backend lives alongside as its own Maven project.

```
onehealth/
├── apps/
│   ├── backend/        # Java Spring Boot API
│   ├── web/            # React web app (patients, doctors, staff)
│   ├── mobile/         # React Native app (patients)
│   └── admin/          # React admin portal (system admin)
├── packages/
│   ├── api-client/     # Generated TS client from OpenAPI spec
│   ├── ui-kit/         # Shared React components (web + admin)
│   ├── types/          # Shared TS types/enums (mirrors backend DTOs)
│   ├── i18n/           # Sinhala/Tamil/English translation bundles
│   └── config/         # Shared ESLint/TS/Tailwind config
├── infra/
│   ├── docker/         # Dockerfiles, docker-compose.yml
│   ├── supabase/       # SQL migrations, RLS policies, seed data
│   └── ci/             # Reusable GitHub Actions workflow fragments
├── docs/                # SRS, ADRs, this booklet, diagrams
├── .github/workflows/   # CI/CD pipelines
├── turbo.json
├── package.json         # npm workspaces root
└── README.md
```

> **Note:** *If a monorepo feels heavier than you want on day one, apps/backend can live as its own repo and everything under apps/ (JS/TS) as a second repo — the internal folder structures below stay identical either way.*

6. Backend — apps/backend (Java / Spring Boot)

Maven multi-module layout, organised by feature/domain rather than by technical layer, so each actor-facing capability (queue, booking, EMR, payments) is easy to find and own.

```
apps/backend/
├── pom.xml                          # parent POM
├── onehealth-api/                    # REST controllers + OpenAPI config
│   └── src/main/java/lk/onehealth/api/
│       ├── auth/                     # JWT filter, Supabase JWKS verification
│       ├── patient/
│       ├── doctor/
│       ├── dispensary/
│       ├── queue/                    # live queue endpoints
│       ├── booking/
│       ├── emr/                      # prescriptions, medical records
│       ├── payment/
│       ├── notification/
│       ├── admin/
│       └── recommendation/           # AI doctor-recommendation endpoints
├── onehealth-core/                   # domain services, business rules
│   └── src/main/java/lk/onehealth/core/
│       ├── queue/QueueService.java
│       ├── booking/BookingService.java
│       ├── emr/EmrService.java
│       ├── payment/PaymentService.java
│       ├── verification/CredentialVerificationService.java
│       └── recommendation/RecommendationEngine.java
├── onehealth-data/                   # JPA entities & repositories
│   └── src/main/java/lk/onehealth/data/
│       ├── entity/
│       ├── repository/
│       └── config/DataSourceConfig.java
├── onehealth-integration/            # external systems
│   └── src/main/java/lk/onehealth/integration/
│       ├── supabase/                 # Storage + Realtime helper clients
│       ├── payhere/
│       ├── slmc/                     # credential registry check
│       └── messaging/                # FCM + SMS gateway
├── onehealth-common/                 # DTOs, exceptions, shared utils, i18n message keys
└── src/test/                          # module-level integration + unit tests
```

7. Web App — apps/web (React + Vite + TypeScript)

Feature-folder structure so Patient, Doctor, and Staff experiences (which share the same app shell but different routes/permissions) stay easy to navigate.

```
apps/web/
├── index.html
├── vite.config.ts
├── src/
│   ├── app/
│   │   ├── routes/                  # route definitions per role
│   │   ├── providers/               # QueryClient, AuthProvider, i18nProvider
│   │   └── App.tsx
│   ├── features/
│   │   ├── auth/
│   │   ├── search-doctors/
│   │   ├── queue/                   # live queue view (subscribes to Supabase Realtime)
│   │   ├── booking/
│   │   ├── profile/
│   │   ├── emr/
│   │   ├── payments/
│   │   └── staff-console/           # staff-only queue management screens
│   ├── components/                  # app-specific components (uses packages/ui-kit)
│   ├── hooks/
│   ├── lib/
│   │   ├── api.ts                   # wraps packages/api-client
│   │   └── supabaseClient.ts        # anon-key client for realtime + storage reads
│   ├── locales/ -> packages/i18n
│   └── main.tsx
├── public/
└── package.json
```

8. Mobile App — apps/mobile (React Native + Expo)

Mirrors the web feature folders where the domain logic is shared (queue, booking, EMR) so the two developers can port a feature between Web and Mobile with minimal re-thinking.

```
apps/mobile/
├── app.json                          # Expo config
├── src/
│   ├── navigation/                   # React Navigation stacks/tabs
│   ├── features/
│   │   ├── auth/
│   │   ├── search-doctors/
│   │   ├── queue/                    # push + realtime queue position
│   │   ├── booking/
│   │   ├── profile/
│   │   ├── emr/
│   │   └── payments/
│   ├── components/                   # NativeWind-styled components
│   ├── hooks/
│   ├── lib/
│   │   ├── api.ts
│   │   ├── supabaseClient.ts
│   │   └── push.ts                   # FCM registration
│   └── App.tsx
├── assets/
└── package.json
```

9. Admin Portal — apps/admin (React + Vite + TypeScript)

Kept as its own app rather than a role inside apps/web, since System Admin covers different concerns (credential approval queue, subscription/tier management, platform analytics, dispute handling) and benefits from a locked-down separate deployment.

```
apps/admin/
├── vite.config.ts
├── src/
│   ├── app/
│   │   ├── routes/
│   │   └── providers/
│   ├── features/
│   │   ├── doctor-verification/      # SLMC credential review queue
│   │   ├── dispensary-management/
│   │   ├── subscriptions/            # Free/Standard/Premium tier admin
│   │   ├── users/
│   │   ├── analytics/
│   │   ├── payments-oversight/
│   │   └── content-moderation/       # feedback/reviews feeding the AI engine
│   ├── components/                   # from packages/ui-kit
│   ├── lib/api.ts
│   └── main.tsx
└── package.json
```

10. Shared Packages

Package	Contents	Consumed by
api-client	TS client auto-generated from the backend's OpenAPI spec (openapi-typescript-codegen)	web, mobile, admin
types	Shared enums/interfaces mirroring backend DTOs (Role, QueueStatus, SubscriptionTier, etc.)	web, mobile, admin, api-client
ui-kit	Buttons, forms, cards, queue-position widget, tier badges — Tailwind-based	web, admin (mobile uses a NativeWind port where practical)
i18n	Sinhala / Tamil / English JSON bundles + i18next config	web, mobile, admin
config	Shared ESLint, TypeScript, Tailwind, and Prettier configs	all JS/TS apps

> **Note:** *Regenerate packages/api-client whenever the backend's OpenAPI spec changes (a CI job can do this automatically) so Web, Mobile, and Admin never hand-write request/response types.*

11. API & Real-Time Communication Conventions

11.1 REST conventions

- Base path: /api/v1/... versioned from day one
- Resource-oriented routes, e.g. GET /api/v1/dispensaries/{id}/queue, POST /api/v1/bookings
- Auth: Authorization: Bearer <supabase-jwt> on every call; backend verifies + resolves role
- Errors: a consistent { code, message, details } envelope so all three clients can share one error-handling hook

11.2 Realtime channels

- queue:{dispensaryId}:{sessionId} — position updates, subscribed by clients currently queued
- Backend writes the authoritative queue state to Postgres; Supabase Realtime pushes the change; clients never infer position from polling

11.3 Idempotency & payments

- Payment-initiating endpoints accept an Idempotency-Key header to avoid duplicate charges on retry — mobile networks in particular make this necessary

12. Environments, CI/CD & Deployment

Environment	Backend	Web/Admin	Mobile	Supabase project
Local	docker-compose (Spring Boot + local Postgres or Supabase CLI)	vite dev server	Expo Go	Supabase CLI local stack
Staging	Fly.io/Render staging service	Vercel/Netlify preview	Expo EAS internal build	Supabase staging project
Production	Fly.io/Render production service	Vercel/Netlify production	EAS production build → App/Play store	Supabase production project

12.1 CI/CD pipeline (GitHub Actions)

- On PR: lint + unit tests for the changed app(s) only (Turborepo's affected-graph keeps this fast for 2 devs)
- On merge to develop: deploy backend + web + admin to staging; run Supabase migration check
- On merge to main (tag release): deploy to production, trigger an EAS build for mobile
- Supabase schema changes are plain SQL migration files under infra/supabase/migrations, applied via the Supabase CLI in CI — never hand-edited in the dashboard for anything beyond local experiments

12.2 Branching model

Continue with the trunk-based flow from the earlier annex: short-lived feature branches off develop, PR review required, main only receives tagged releases from develop.

13. Developer Onboarding Checklist

- Clone the monorepo, run npm install at the root (workspaces install everything under apps/ and packages/)
- Install the Supabase CLI, run supabase start for a local Postgres + Auth + Storage stack
- Copy .env.example to .env in each app; fill in the local Supabase URL/anon key and backend base URL
- Build the backend: cd apps/backend && mvn spring-boot:run (module onehealth-api)
- Run web: npm run dev -w apps/web · admin: npm run dev -w apps/admin · mobile: npm run start -w apps/mobile (Expo)
- Run infra/supabase/migrations against the local stack to get schema + seed data
- Confirm the OpenAPI spec regenerates packages/api-client cleanly (npm run generate:api at root)
- Read docs/ARCHITECTURE.md and the SRS before picking up a feature ticket

> **Note:** *Keep this checklist current — the two-developer setup only stays fast if a fresh clone can be running locally in under 30 minutes.*